

Introduction to Artificial Intelligence

Constraint Satisfaction Problems

Matteo Acclavio

12/03/2026

Plan

So far:

- We have seen how to describe an agent and its environment
- We have seen how to plan by considering the state as 'atomic'

Today:

- Describe a problem as a set of variables and constraints
- Solution is a configuration satisfying all constraints
- CSP

The main idea

- Factoring the state into variables

The main idea

- Factoring the state into variables
- Add constraints to specify which variable combinations are allowed

The main idea

- Factoring the state into variables
- Add constraints to specify which variable combinations are allowed
- Solution = assignment of values to variables that satisfies all constraints

The main idea

Examples of problems easy to model as CSPs:

- Sudoku

The main idea

Examples of problems easy to model as CSPs:

- Sudoku
- Map-coloring

The main idea

Examples of problems easy to model as CSPs:

- Sudoku
- Map-coloring
- N-queens

The main idea

Examples of problems easy to model as CSPs:

- Sudoku
- Map-coloring
- N-queens
- Class scheduling

A ***solution*** is a state that satisfies all constraints.

Interlude: how to solve problems?

Imperative vs. Declarative Approaches

Goal: find a solution to a problem

Imperative approach:

- Explicitly describes *how* to compute a solution
- Requires designing and implementing a search strategy
- Example (Sudoku): write a search algorithm to check every possibility

Declarative approach:

- Specifies *what* constraints a solution must satisfy
- Solver automatically finds feasible solutions
- Example (Sudoku): state the constraints — “each row/column/block contains 1–9 exactly once”

Constraint satisfaction problems

Formally

A **constraint satisfaction problem** (CSP) consists of three components:

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$

Formally

A **constraint satisfaction problem** (CSP) consists of three components:

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$

Formally

A **constraint satisfaction problem** (CSP) consists of three components:

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Formally

A **constraint satisfaction problem** (CSP) consists of three components:

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

An **assignment** (=state of the system) is a set of variable-value pairs

$$\{X_{i_1}, \dots, X_{i_k} = v_{i_1}, \dots, v_{i_k}\} \text{ with } \{i_1, \dots, i_k\} \subseteq \{1, \dots, n\}$$

Formally

A **constraint satisfaction problem** (CSP) consists of three components:

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

An **assignment** (=state of the system) is a set of variable-value pairs

$$\{X_{i_1}, \dots, X_{i_k} = v_{i_1}, \dots, v_{i_k}\} \text{ with } \{i_1, \dots, i_k\} \subseteq \{1, \dots, n\}$$

- It is **partial** if $I \neq \{1, \dots, n\}$, and **complete** otherwise.

Formally

A **constraint satisfaction problem** (CSP) consists of three components:

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

An **assignment** (=state of the system) is a set of variable-value pairs

$$\{X_{i_1}, \dots, X_{i_k} = v_{i_1}, \dots, v_{i_k}\} \text{ with } \{i_1, \dots, i_k\} \subseteq \{1, \dots, n\}$$

- It is **partial** if $I \neq \{1, \dots, n\}$, and **complete** otherwise.
- It is **consistent** if it satisfies all constraints.

Formally

A **constraint satisfaction problem** (CSP) consists of three components:

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

An **assignment** (=state of the system) is a set of variable-value pairs

$$\{X_{i_1}, \dots, X_{i_k} = v_{i_1}, \dots, v_{i_k}\} \text{ with } \{i_1, \dots, i_k\} \subseteq \{1, \dots, n\}$$

- It is **partial** if $I \neq \{1, \dots, n\}$, and **complete** otherwise.
- It is **consistent** if it satisfies all constraints.
- It is a **solution** if it is complete and consistent.

Example: Sudoku

- A set of **variables** $\mathcal{X}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (sudoku)

Example: Sudoku

- A set of **variables** $\mathcal{X}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (sudoku)

- Variables: X_{ij} with $i, j \in \{1, \dots, 9\}$

Example: Sudoku

- A set of **variables** $\mathcal{X}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (sudoku)

- Variables: X_{ij} with $i, j \in \{1, \dots, 9\}$
- Domains: $D_{ij} = \{1, \dots, 9\}$ for each variable $i, j \in \{1, \dots, 9\}$
- Constraints:
 - Row constraints:

Example: Sudoku

- A set of **variables** $\mathcal{X}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (sudoku)

- Variables: X_{ij} with $i, j \in \{1, \dots, 9\}$
- Domains: $D_{ij} = \{1, \dots, 9\}$ for each variable $i, j \in \{1, \dots, 9\}$
- Constraints:
 - Row constraints: $X_{i1}, \dots, X_{i9}$ are all different for each $i \in \{1, \dots, 9\}$
 - Column constraints:

Example: Sudoku

- A set of **variables** $\mathcal{X}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (sudoku)

- Variables: X_{ij} with $i, j \in \{1, \dots, 9\}$
- Domains: $D_{ij} = \{1, \dots, 9\}$ for each variable $i, j \in \{1, \dots, 9\}$
- Constraints:
 - Row constraints: $X_{i1}, \dots, X_{i9}$ are all different for each $i \in \{1, \dots, 9\}$
 - Column constraints: $X_{1j}, \dots, X_{9j}$ are all different for each $j \in \{1, \dots, 9\}$
 - Block constraints:

Example: Sudoku

- A set of **variables** $\mathcal{X}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (sudoku)

- Variables: X_{ij} with $i, j \in \{1, \dots, 9\}$
- Domains: $D_{ij} = \{1, \dots, 9\}$ for each variable $i, j \in \{1, \dots, 9\}$
- Constraints:
 - Row constraints: $X_{i1}, \dots, X_{i9}$ are all different for each $i \in \{1, \dots, 9\}$
 - Column constraints: $X_{1j}, \dots, X_{9j}$ are all different for each $j \in \{1, \dots, 9\}$
 - Block constraints:
 $X_{3k+1,3l+1}, X_{3k+1,3l+2}, X_{3k+1,3l+3}, X_{3k+2,3l+1}, X_{3k+2,3l+2}, X_{3k+2,3l+3}, X_{3k+3,3l+1},$
are all different for each $k, l \in \{0, \dots, 2\}$

Example: Map coloring

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (Map coloring)

Example: Map coloring

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (Map coloring)

- Variables: $X_{\text{state name}}$ for each state

Example: Map coloring

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (Map coloring)

- Variables: $X_{\text{state name}}$ for each state
- Domains: $D_{\text{state name}} = \{\text{red, blue, green}\}$ for each state
- Constraints:

Example: Map coloring

- A set of **variables** $\mathcal{X} = \{X_i \mid i \in I\}$
- A **domain** D_X of possible values for each variable $X \in \mathcal{X}$
- A set of **constraints** specifying allowable combinations of values

Example (Map coloring)

- Variables: $X_{\text{state name}}$ for each state
- Domains: $D_{\text{state name}} = \{\text{red, blue, green}\}$ for each state
- Constraints: adjacent states must have different colors
(example: $X_{\text{Denmark}} \neq X_{\text{Germany}}$)

The Constraint Graph of a CSP

A CSP can be represented as a **constraint graph**

The Constraint Graph of a CSP

A CSP can be represented as a **constraint graph**

- Nodes are variables

The Constraint Graph of a CSP

A CSP can be represented as a **constraint graph**

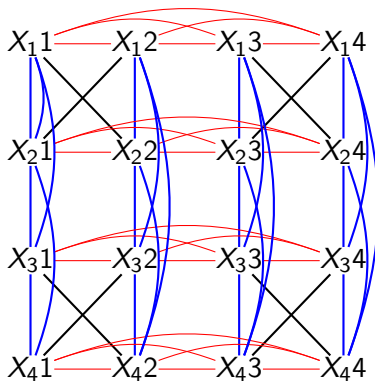
- Nodes are variables
- Edges/arcs show constraints

The Constraint Graph of a CSP

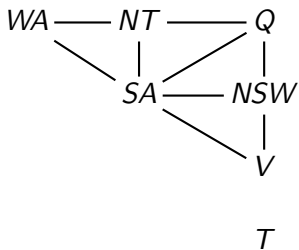
A CSP can be represented as a **constraint graph**

- Nodes are variables
- Hyper-edges show constraints

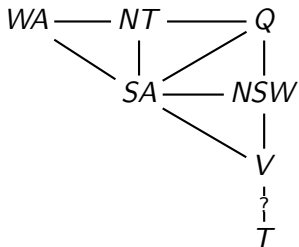
CSP graph for 2-Sudoku



CSP graph for coloring Australia

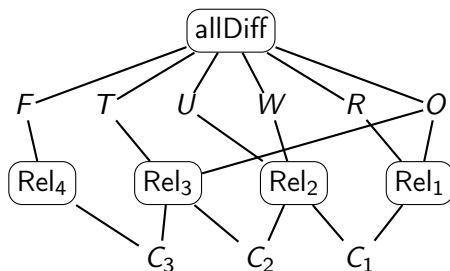


CSP graph for coloring Australia



CSP graph for Letters are numbers

$\begin{array}{rcccc} T & W & O & + \\ T & W & O & = \\ \hline F & O & U & R \end{array}$	$\begin{array}{l} \text{Rel}_1 \\ \text{Rel}_2 \\ \text{Rel}_3 \\ \text{Rel}_4 \\ \text{AllDiff} \end{array}$	$\begin{array}{l} O + O = R + 10C_1 \\ W + W + C_1 = U + 10C_2 \\ T + T + C_2 = O + 10C_3 \\ C_3 = F \end{array}$
---	---	---



Solving CSPs

Approach 1: Standard Search Formulation

- **Initial state:** empty assignment $\{ \}$
- **Successor function:** assign a value to an unassigned variable that does not conflict with the current assignment
- If all variables are assigned $\rightarrow$ problem solved
- If some variable cannot be assigned $\rightarrow$ backtrack and try another value
- If we backtrack past the first variable $\rightarrow$ problem is unsatisfiable

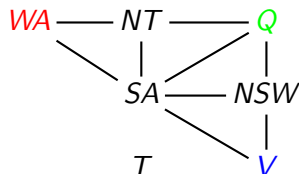
Backtracking

Algorithm 1 Backtracking Search

```
function BACKTRACKING-SEARCH(csp)  
    return BACKTRACK({}, csp)  
end function  
function BACKTRACK(assignment, csp)  
    if assignment is complete then return assignment  
    end if  
    var  $\leftarrow$  SelectUnassignedVariable(csp)  
    for each val in OrderDomainValues(var, csp) do  
        if val is consistent with assignment then  
            add {var = val} to assignment  
            result  $\leftarrow$  BACKTRACK(assignment, csp)  
            if result  $\neq$  failure then return result  
            end if  
            remove {var = val} from assignment  
        end if  
    end for
```

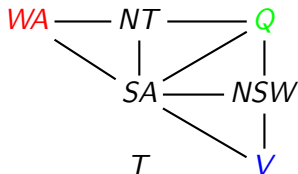

Backtracking example (coloring Australia)

- $X = \{WA, NT, Q, NSW, V, SA, T\}$
- $D_X = \{R, G, B\}$ for each variable
- Constraints: adjacent states must have different colors



Backtracking example (coloring Australia)

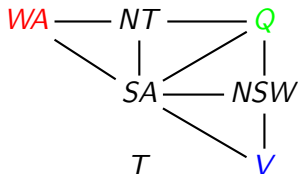
- $X = \{WA, NT, Q, NSW, V, SA, T\}$
- $D_X = \{R, G, B\}$ for each variable
- Constraints: adjacent states must have different colors



	WA	NT	Q	NSW	V	SA	T
Initial	?	?	?	?	?	?	?

Backtracking example (coloring Australia)

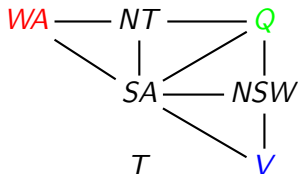
- $X = \{WA, NT, Q, NSW, V, SA, T\}$
- $D_X = \{R, G, B\}$ for each variable
- Constraints: adjacent states must have different colors



	WA	NT	Q	NSW	V	SA	T
Initial	?	?	?	?	?	?	?
WA=R	R	?	?	?	?	?	?

Backtracking example (coloring Australia)

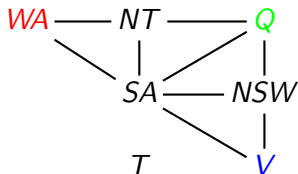
- $X = \{WA, NT, Q, NSW, V, SA, T\}$
- $D_X = \{R, G, B\}$ for each variable
- Constraints: adjacent states must have different colors



	WA	NT	Q	NSW	V	SA	T
Initial	?	?	?	?	?	?	?
WA=R	R	?	?	?	?	?	?
Q=G	R	?	G	?	?	?	?

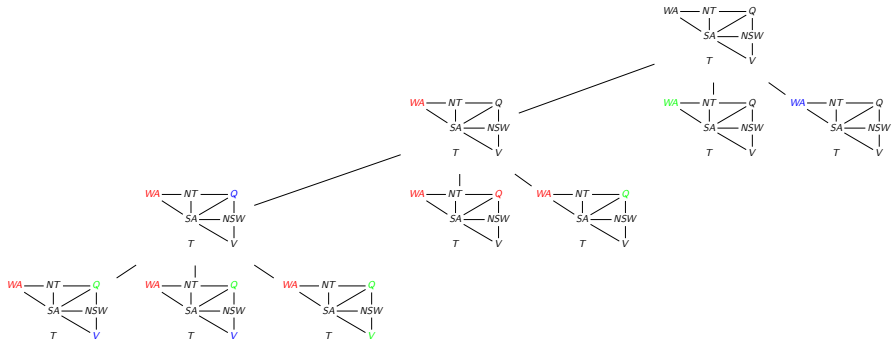
Backtracking example (coloring Australia)

- $X = \{WA, NT, Q, NSW, V, SA, T\}$
- $D_X = \{R, G, B\}$ for each variable
- Constraints: adjacent states must have different colors



	WA	NT	Q	NSW	V	SA	T
Initial	?	?	?	?	?	?	?
WA=R	R	?	?	?	?	?	?
Q=G	R	?	G	?	?	?	?
V=B	R	?	G	?	B	?	?

Backtracking example (coloring Australia) tree



Constraint Propagation

Improving Backtracking Efficiency

General-purpose methods can give huge gains in speed:

- Can we detect inevitable failure early?

Improving Backtracking Efficiency

General-purpose methods can give huge gains in speed:

- Can we detect inevitable failure early? → **constraint propagation**
- Which variable should be assigned next?

Improving Backtracking Efficiency

General-purpose methods can give huge gains in speed:

- Can we detect inevitable failure early? → **constraint propagation**
- Which variable should be assigned next? → **variable heuristics**
- In what order should its values be tried?

Improving Backtracking Efficiency

General-purpose methods can give huge gains in speed:

- Can we detect inevitable failure early? → **constraint propagation**
- Which variable should be assigned next? → **variable heuristics**
- In what order should its values be tried? → **value heuristics**

Constraint propagation

Constraint propagation is based on the idea of **local consistency**: a variable is locally consistent if it satisfies some consistency condition with respect to its neighbors in the constraint graph.

- Node Consistency (NC)
- Arc Consistency (AC)
- Path Consistency (PC)
- k -Consistency
- Strong k -Consistency

Node Consistency

Not really “propagation”, but a useful preprocessing step: remove values from variable domains that violate unary constraints.

Definition: Node Consistency

X is **node consistent** iff for every value $x \in D_X$, x satisfies the unary constraints on X .

- Easy to enforce: just check each value in the domain of each variable
- Can be run as a preprocessor or interleaved with backtracking
- Can be avoided by restricting the initial domains

Arc Consistency

Simplest form of propagation: make each arc consistent.

Definition: Arc Consistency

A variable X is *arc consistent* wrt Y if for any $x \in D_X$ there is a $y \in D_Y$ such that (x, y) satisfy any binary constraint on X and Y .

Example: $D_X = D_Y = \{0, \dots, 9\}$ and consider the constraint $Y = 2X$.

- is X arc consistent w.r.t. Y ?
- is Y arc consistent w.r.t. X ?
- is there a way to fix this?

Arc Consistency in action

	WA	NT	Q	NSW	V	SA	T
Initial	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B
$WA=R$	R	G,B	R,G,B	R,G,B	R,G,B	G,B	R,G,B
$Q=G$	R	B	G	R,B	R,G,B	B	R,G,B
$V=B$	R	B	G	R	B	—	R,G,B

Arc Consistency in action

	WA	NT	Q	NSW	V	SA	T
Initial	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B
WA=R	R	G,B	R,G,B	R,G,B	R,G,B	G,B	R,G,B
Q=G	R	B	G	R,B	R,G,B	B	R,G,B
V=B	R	B	G	R	B	—	R,G,B

- 1 After $V=B$: SA has no legal values $\Rightarrow$ **failure detected early!**
- 2 Arc consistency can prune domains before and during search.
- 3 May not be capable of detecting all failures

Path Consistency

One step beyond!

Definition: Path Consistency

A pair of variables $\{X, Y\}$ is ***path consistent*** wrt a third variable Z if for every assignment to X and Y which is consistent (wrt any constraint on X and Y), there is an assignment to Z that satisfies the constraints between X and Z and between Y and Z .

Example

- Variables: X , Y , and Z
- Domains: $D_X = D_Y = D_Z = \{1, 2, 3\}$
- Constraints: $X < Y$, $Y < Z$, $X < Z$

k -Consistency

Definition: k -Consistency

A set of k variables $\{X_1, \dots, X_k\}$ is **k -consistent** if for any consistent assignment to any $k - 1$ of these variables, there is an assignment to the k -th variable that satisfies all constraints.

Example (Almost finished Sudoku)

Any n -Sudoku game with 1 empty cell is n^2 -consistent.

Strong k -Consistency

Definition: Strong k -Consistency

A set of k variables $\{X_1, \dots, X_k\}$ is ***strong k -consistent*** iff it is j -consistent for all $j \leq k$.

Strong k -Consistency

Definition: Strong k -Consistency

A set of k variables $\{X_1, \dots, X_k\}$ is **strong k -consistent** iff it is j -consistent for all $j \leq k$.

Note: k -consistency does not imply $k + 1$ -consistency!

Example (Latin Square)

- Variables: X_{ij} for $i, j \in \{1, \dots, 3\}$
- Domains: $D_{ij} = \{A, B, C\}$ for each i, j
- Constraints: all different in each row and column

Strong k -Consistency

Definition: Strong k -Consistency

A set of k variables $\{X_1, \dots, X_k\}$ is **strong k -consistent** iff it is j -consistent for all $j \leq k$.

Note: k -consistency does not imply $k + 1$ -consistency!

Example (Latin Square)

- Variables: X_{ij} for $i, j \in \{1, \dots, 3\}$
- Domains: $D_{ij} = \{A, B, C\}$ for each i, j
- Constraints: all different in each row and column

We have:

- n^2 -consistency for any n

Strong k -Consistency

Definition: Strong k -Consistency

A set of k variables $\{X_1, \dots, X_k\}$ is **strong k -consistent** iff it is j -consistent for all $j \leq k$.

Note: k -consistency does not imply $k + 1$ -consistency!

Example (Latin Square)

- Variables: X_{ij} for $i, j \in \{1, \dots, 3\}$
- Domains: $D_{ij} = \{A, B, C\}$ for each i, j
- Constraints: all different in each row and column

We have:

- n^2 -consistency for any n
- not necessary k -consistent for $k < n^2$ (example $n = 2$ and

A	
	B

Heuristics

Value Heuristics

Least Constraining Value (LCV):

- Choose the value that eliminates the fewest options for other variables
- Keeps more flexibility in the search space

Most Constraining Value:

- Pick the value that rules out the most other values
- Helps detect failure earlier

Min-Value / Max-Value:

- Pick the smallest (or largest) value first
- Useful when values have a natural order (e.g. cost minimization)

Variable Heuristics

Minimum Remaining Values (MRV):

- Pick the variable with the *smallest* domain (“fail-first”)
- Quickly reduces the search space

Degree Heuristic:

- Pick the variable involved in the most constraints
- Useful as a tie-breaker for MRV

Dom/Wdeg (adaptive):

- Weighted degree = sum of constraint weights based on past failures
- Choose variable minimizing $\frac{\text{domain size}}{\text{weighted degree}}$

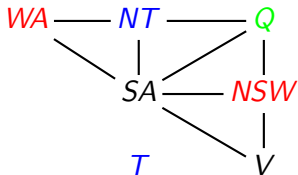
Advanced Techniques

Backjumping

Improved form of backtracking that **jumps back multiple levels**:

- Analyses *why* the failure occurred
- Jumps directly to the most relevant earlier decision

Example:



Summary

Book references:

- Chapter 5.1, 5.2, 5.3, 5.4

Next TA session:

- More exercises on search [Chapter 3 and 4 so far]

Next lecture:

- Representing problems using logic